***Danny Beaudoin***

***SFWRTECH 4SA3 – Project***

***Milestone 3 – Software Architecture Document (4 + 1 View Model)***

***14 March 2026***

## Table of Contents

1. Project Purpose & Audience: ..... 3
2. Changes Made: ..... 3
3. Requirements: ..... 4
3.1 Requirement table ..... 4
4. Scenario Viewpoint: ..... 5
4.1 Use Case Diagram ..... 5
4.2 Diagram description ..... 5
5. Physical Viewpoint: ..... 6
5.1 System Deployment Diagram ..... 6
5.2 Deployment diagram description ..... 6
6. Development Viewpoint: ..... 8
6.1 System class diagram ..... 8
6.2 class diagram description ..... 8
7. Logical Viewpoint: ..... 9
7.1 Delivery object state diagram ..... 9
7.2 state diagram description: ..... 9
8.1 route retrieval process sequence diagram ..... 11
8.2 sequence diagram description: ..... 11

## 1. Project Purpose & Audience:

Having spent many years working in the fast-food industry, specifically in pizzerias, I have noticed a recurring problem when it comes to delegating deliveries to drivers during peak business hours. The goal of this software is to allow a dispatcher to go into this application and be able to enter in all the deliveries that they need to send out. The application will then calculate and display the “most efficient” route to take with all the deliveries. The dispatcher can then manage these deliveries with various functions. With that being said, the target audience of this application is generally going to be fast-food restaurant operators, specifically pizzeria managers in this case.

## 2. Changes Made:

Since having submitted the last milestone, I have spoken to the instructor and received permission to modify the architecture of my assignment, reducing scope to save time. I am no longer going to be using React for the front end. I have decided to use Django Templates to serve the UI to the client. The deployment diagram has been modified to reflect these changes.

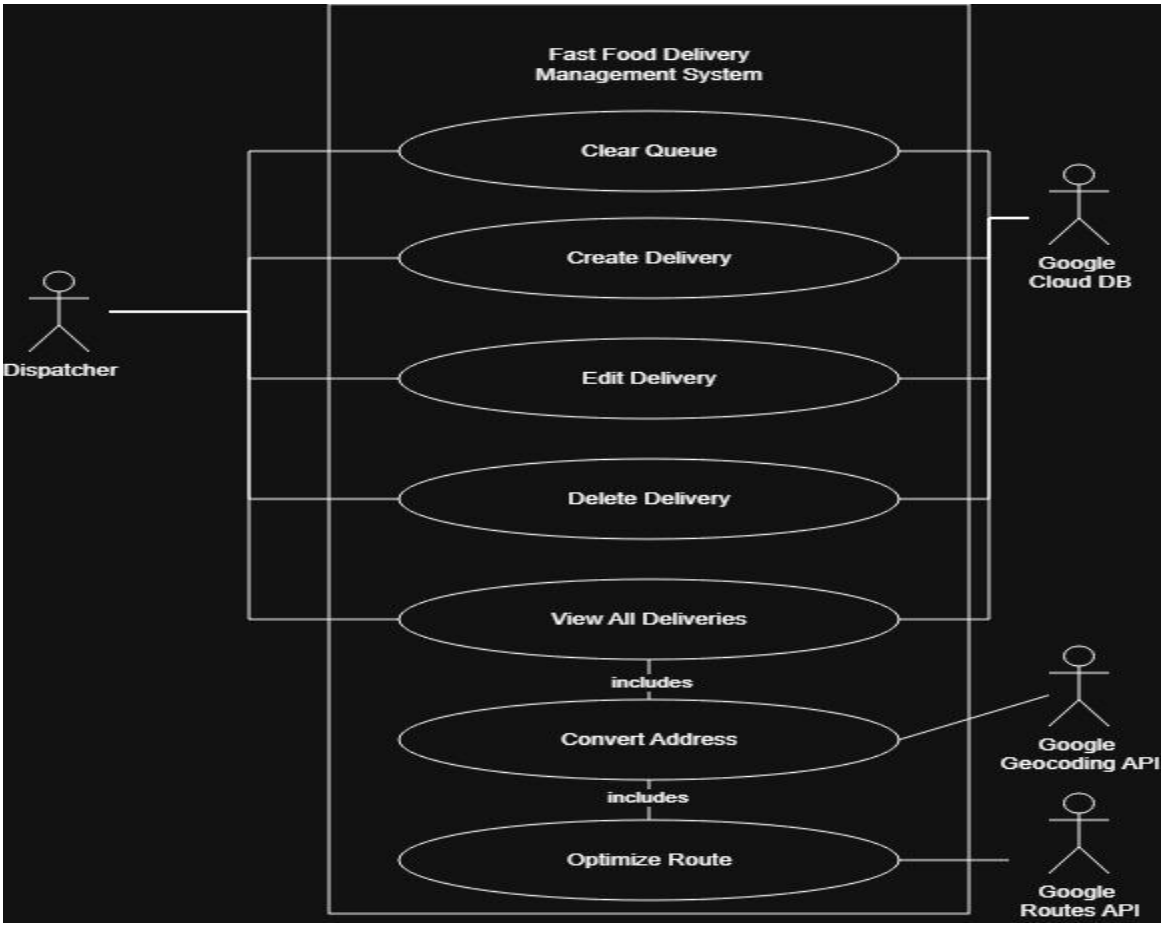
# 3. Requirements:

### 3.1 Requirement table

| Name | Type | Description |
|---|---|---|
| Create Delivery | Functional | The system will allow a dispatcher to create a new delivery |
| Edit Delivery | Functional | The system will allow a dispatcher to edit an existing delivery |
| Delete Delivery | Functional | The system will allow a dispatcher to delete an existing delivery |
| View All Deliveries | Functional | The system shall display all current deliveries in a queue on the main screen |
| Convert Address | Functional | The system will be able to convert strings in the form of an address into latitude / longitude coordinates via google geocoding API |
| Optimize Route | Functional | The system shall generate the most optimum route for a set of queued deliveries using googles Routes API |
| Clear Queue Functionality | Functional | The system must have an option that allows the dispatcher to clear all deliveries from the queue |
| Scalability | Non-functional | The system shall support multiple restaurant locations without requiring major architectural redesign |
| Reliability | Non-functional | The system shall handle third party API failures gracefully and display useful error messages |
| Security | Non-functional | The system must securely store delivery data in a MySQL database hosted on Google Cloud and protect API keys from exposure |

# 4. Scenario Viewpoint:

### 4.1 Use Case Diagram




### 4.2 Diagram description

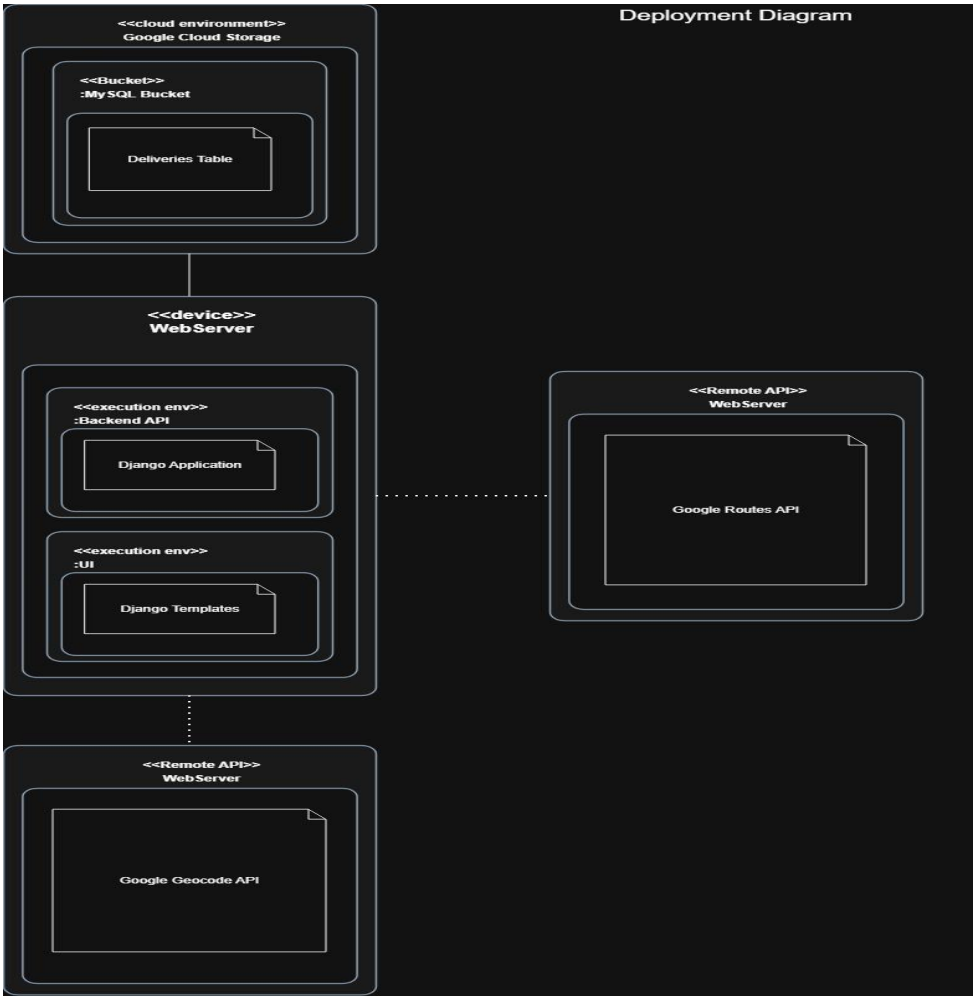
Above is a use case diagram which depicts the functional requirements of the application. It contains 7 different system components which each map to a corresponding functional requirement in the requirements table. The basic CRUD operations that are available to the dispatcher are to create a delivery to be added to queue, edit a delivery in the queue, delete a delivery, delete all deliveries, and view all deliveries. All of the above pieces of functionality also require the application to interact with the Google Cloud Database that we have setup, so you can see the appropriate relationship in the diagram.

The View All Deliveries function will require the convert address functionality to be carried out, which also interacts with the Google Geocoding API. This is marked with a relationship mapped to the actor. This is where the address that the user entered gets converted into coordinates to be passed on to the next component of the system.

Lastly, the system requires the Optimize Route function to be carried out as well, given the coordinates obtained in the previous step. This step requires the use of the Google Routes API, as indicated via the relationship in the diagram.

## 5. Physical Viewpoint:

### 5.1 System Deployment Diagram




### 5.2 Deployment diagram description

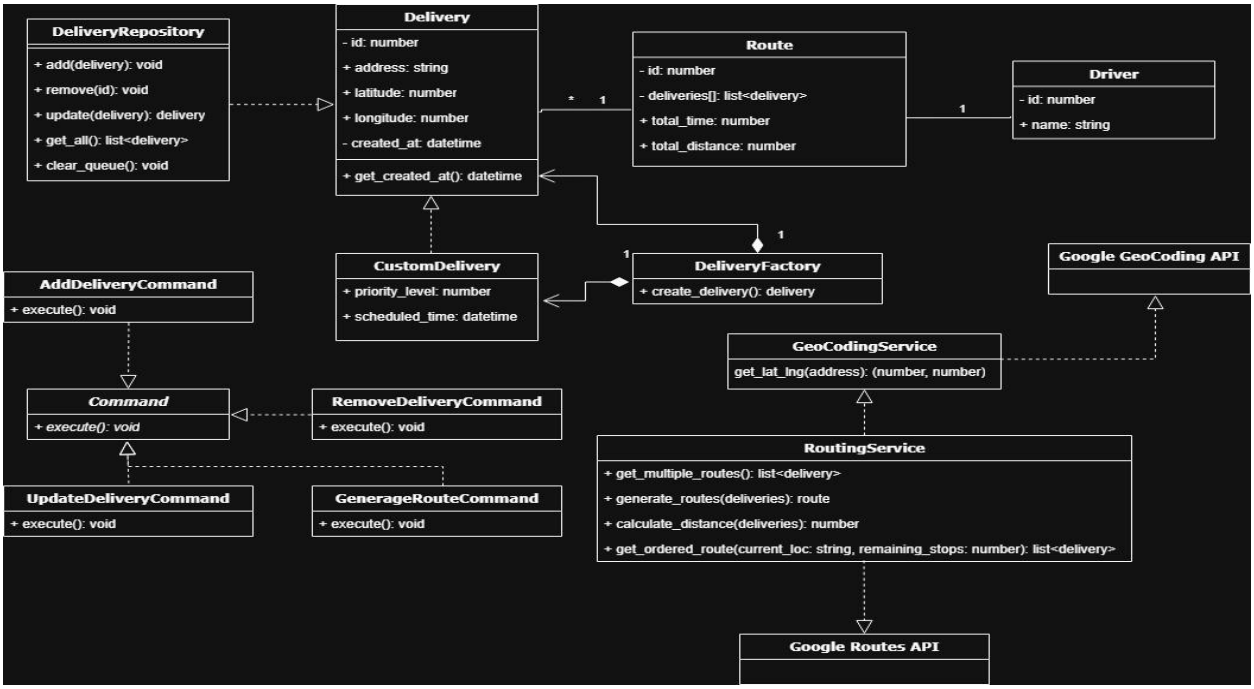
Above is a deployment diagram for the system. As previously mentioned, the system will consist of 4 main components each represented by a component image. Some of the components contain child components. Any of the external APIs do not due to the lack of public information.

The User Interface in this case is just going to be Django templates, which will use Django to serve the webpages combined with the data retrieved and processes in the API.

The back-end API also passes information to and from Google Cloud SQL which contains our MySQL database. It also contains two relationships. One between our back-end API and Google's Routes API, and the other between our API and Google Geocoding API. Our Django API runs inside a Django execution environment which is hosted on a backend webserver.

# 6. Development Viewpoint:

### 6.1 System class diagram




### 6.2 class diagram description

The above class diagram models my Restaurant Delivery Management System and all of its applicable classes. The diagram shows 16 different classes that are relevant to my application. 2 of these classes are external APIs setup by google. They are Google Routes API and Google GeoCoding API.

The main part of the application is going to be in the RoutingService class. This class will contain references to the external APIs, and it will also handle the business logic for handling and processing delivery routes.

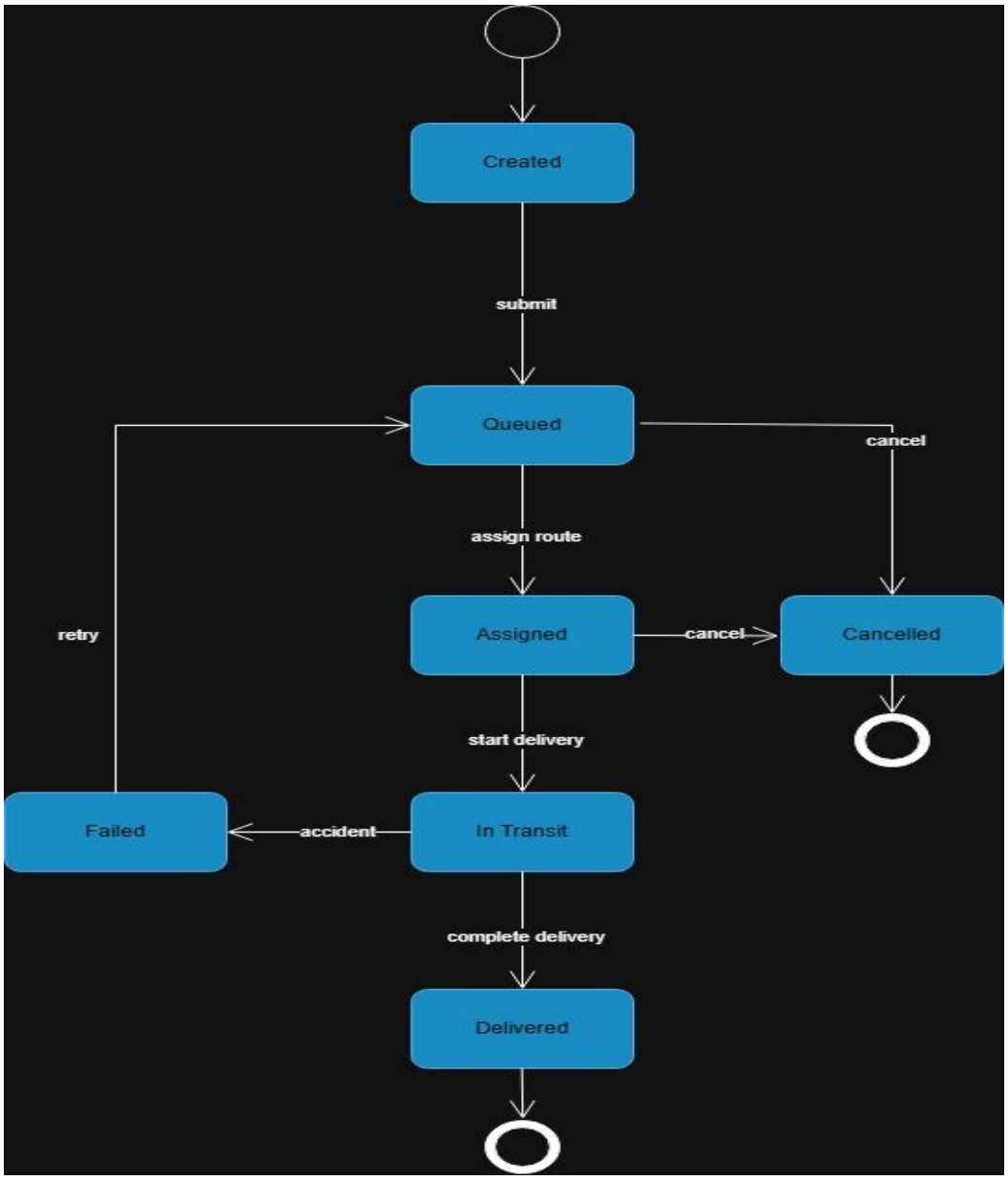
There is also a base Command class with an execute method. The base class is abstract as well as this method. They are to be implemented via the extended classes: UpdateDeliveryCommand, RemoveDeliveryCommand, AddDeliveryCommand, and GenerateRouteCommand. All of these classes extend the base class and therefore need to implement the execute method.

Next, we have our model which is the delivery object. It contains an id, address, lat, long, and created_at timestamp. We also have the CustomDelivery class which extends delivery and adds the ability to schedule timed orders and set the priority of the delivery. Both delivery classes show a relationship with the DeliveryFactory class. Which is used to dynamically create different types of delivery objects at runtime. The Delivery class also has a relationship with the DeliveryRepository class. This is being used to override the

default Django behavior. Lastly, we have a driver object, and a route object. One driver may have one route, and one route can have many deliveries.

## 7. Logical Viewpoint:

### 7.1 Delivery object state diagram



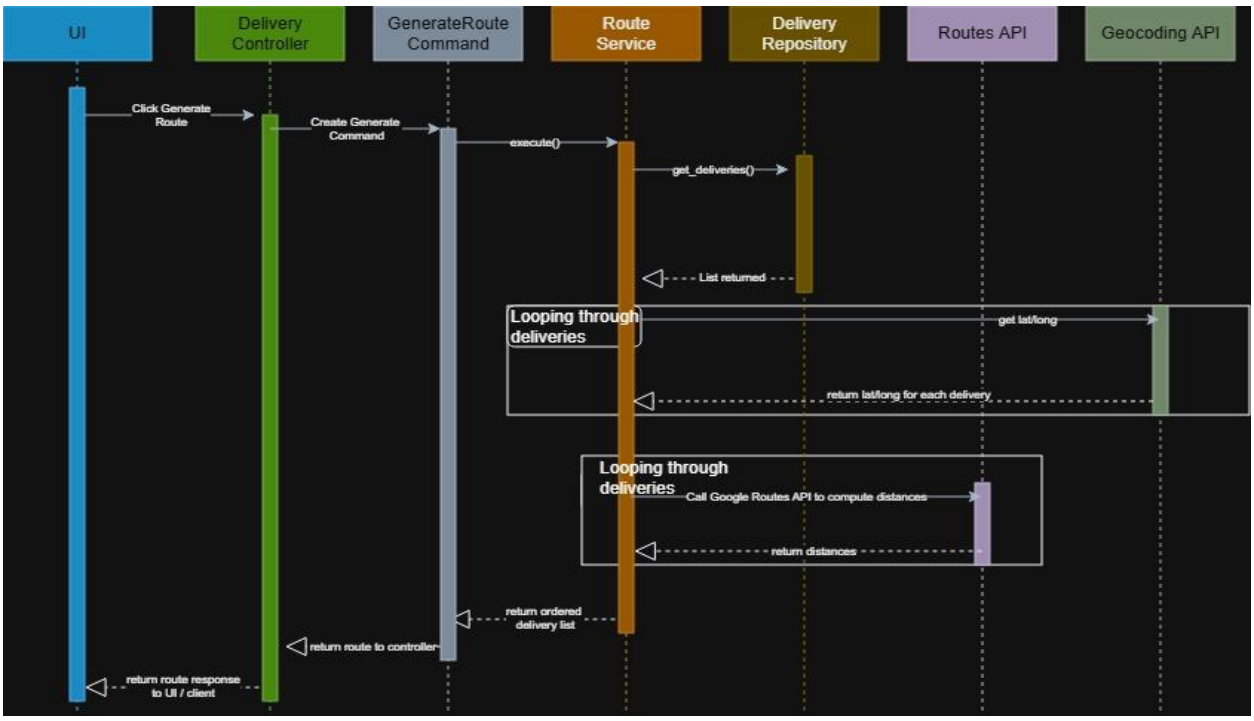

### 7.2 state diagram description:

The above state diagram describes the nature of a delivery and its possible states / activities. A delivery object can have 7 possible states. Created, Queued, Assigned, In Transit, Delivered, Failed and Cancelled. Once a delivery object is created, it then gets submitted to the queue, and the state changes to “queued”. After this it can either be

assigned to a route or cancelled. If the system assigns it to a route, its state will change to “assigned”. If not, it will have a state of “cancelled”. Once the driver starts the delivery, the state will change to “in transit”. From here, if the driver gets into an accident the state of the delivery will change to “failed”. If the delivery is complete, the state will change to “delivered”, and that will end the delivery.

# 8. Process Viewpoint:

### 8.1 route retrieval process sequence diagram




### 8.2 sequence diagram description:

The above sequence diagram illustrates the sequence of events that take place in order for a user to obtain an ordered list of deliveries, aka: a route. There are 7 lifelines in this diagram: UI, DeliveryController, GenerateRouteCommand, RouteService, DeliveryRepository, RoutingService (Google Routes API) and Geocoding Service (Google Geocoding API)

Initially the user will input addresses into the system (not shown here). The user will click the “generate route” button. This will then trigger the delivery controller to create a generate route command. This command will then run it’s execute() method, which will go into the Route Service and call a get_deliveries() method. This method will then go into the DeliveryRepository and call its method to query the DB for delivery objects. This list then gets returned to the repo, and then back to the service.

Next the RouteService will loop through the list of deliveries, calling the Geocoding services get lat/long method and return a list of tuples with lat/long coordinates for each delivery object. Then, the RouteService will call the Google Routes API to compute distances, iterating through the list of coordinates. Lastly, the RouteService will use its business logic to compute the route with the lowest time / distance and return that to the controller, which then gets returned in a responsebody to the UI.